



Guided Coding

Staying in Control While Coding With AI Agents

dodnedder

2026-09-17

Kenny Pflug

On today's menu

- **What is Guided Coding? How does it differ from Vibe Coding?**
 - The three phases: plan, implement, guide
 - How to structure plans
 - How to handle AGENTS.md and Skills
 - With practical examples from my newest OSS library [Light.PortableResults](#)
- tbd

Kenny Pflug

[@feO2x](#)

Engineering Manager
TELIS/GWVS

- C#/.NET since 2009
- Distributed Cloud-Native Systems since 2014
- AI Integration and Coding Agents since 2023
- CLR and Framework Internals
(Memory Management,
Asynchronous Programming, Threading,
ORMs, DI Containers, Serializers, etc.)



The Five Levels

[See The Five Levels: From Spicy Autocomplete to the Dark Factory](#)

SPICY AUTOCOMPLETE (Level 0)



The AI serves as a reference tool or enhanced search, like a smarter Stack Overflow. The developer manually writes all the code, using AI for specific snippets or answers.

CODING INTERN (Level 1)



The AI writes unimportant or boilerplate code. The developer prompts for specific functions but immediately reviews and integrates the output.

JUNIOR DEVELOPER (Level 2)



An interactive 'pair programmer' partnership. The developer and AI trade off control, with the human reviewing code as it's generated in real-time.

DEVELOPER (Level 3)



The AI generates the majority of the codebase. The developer reviews everything the AI does, acting as the bottleneck for verification before progressing.

ENGINEERING TEAM (Level 4)



The AI runs unattended for long periods, handling complex tasks. The human trusts the system's self-checks, only checking the final results much later.

DARK SOFTWARE FACTORY (Level 5)



The engineer manages the goals and the system, not the code. They provide plain English descriptions. The AI defines implementation, writes code, tests, fixes bugs, and ships.

Guided Coding is Level 3.5



Writing Enterprise-Grade Software with Coding Agents

Vibe Coding

- [Originally posted in February 2025](#) by Andrej Karpathy, co-founder of OpenAI, on X
- Develop software with coding agents, but never look at the source code
- Simply “Accept All” code changes, code “grows beyond usual comprehension”, but you get stuff done fast

This term has become a [slang & trending expression according to Merriam Webster](#)

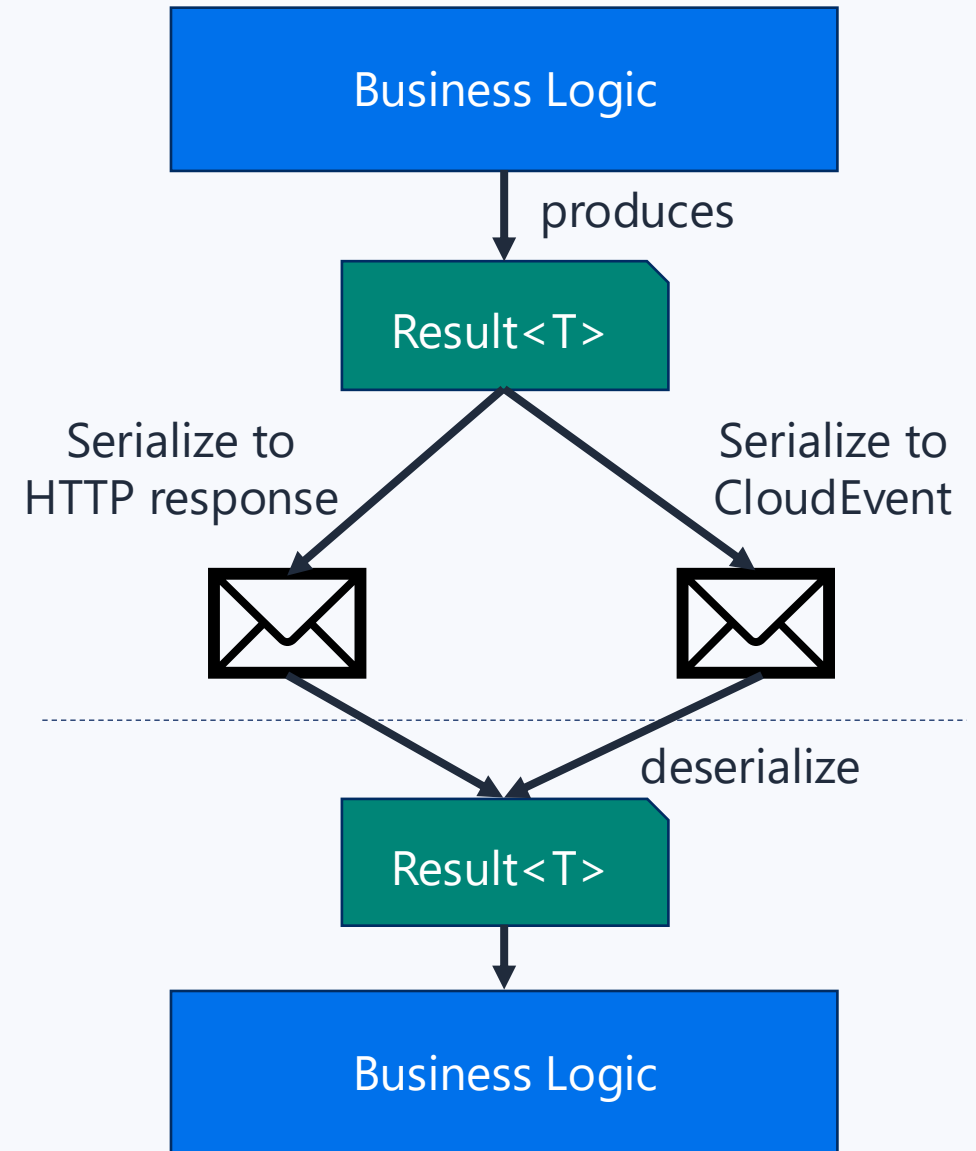
This is not suitable for enterprise-grade software – **but how can we let Coding Agents write most of the software while staying in control?**



Light.PortableResults as a test subject

- Implements the result pattern in .NET, but with a twist: every result is serializable and deserializable and thus can be mapped to any protocol format.
- A result is either a success with a value, or a list of errors. A result and each error can have metadata attached to it.
- Idea: Business Logic creates a Result<T> instance which can be easily mapped to HTTP response, CloudEvent, etc. Deserialize at the receiving end and feed to Business Logic.

Can I write this with an AI-first approach?



Light.PortableResults results

- Started on **January 5th, 2026**, first version 0.1.0 released on **February 25th, 2026**, second version 0.2.0 released **April 13th, 2026**, latest version 0.7.0 released on **August 4th, 2026**.
- What's inside?
 - Core implementation (results, errors, metadata)
 - (de-)serialization to/from HTTP responses, including support for RFC-9457 Problem Details
 - (de-)serialization to/from CloudEvent Spec 1.0 JSON format
 - Integration with ASP.NET Core Minimal APIs and MVC
 - Non-trivial customization for HTTP and CloudEvents
 - Performance-optimized
 - Validation and OpenAPI integration
- What's missing?
 - Protobuf and gRPC support
- `git ls-files '*' | xargs wc -l: 138,168 lines`
`cloc . --include-lang="C#" --exclude-dir=bin,obj: 64,356 lines of code`

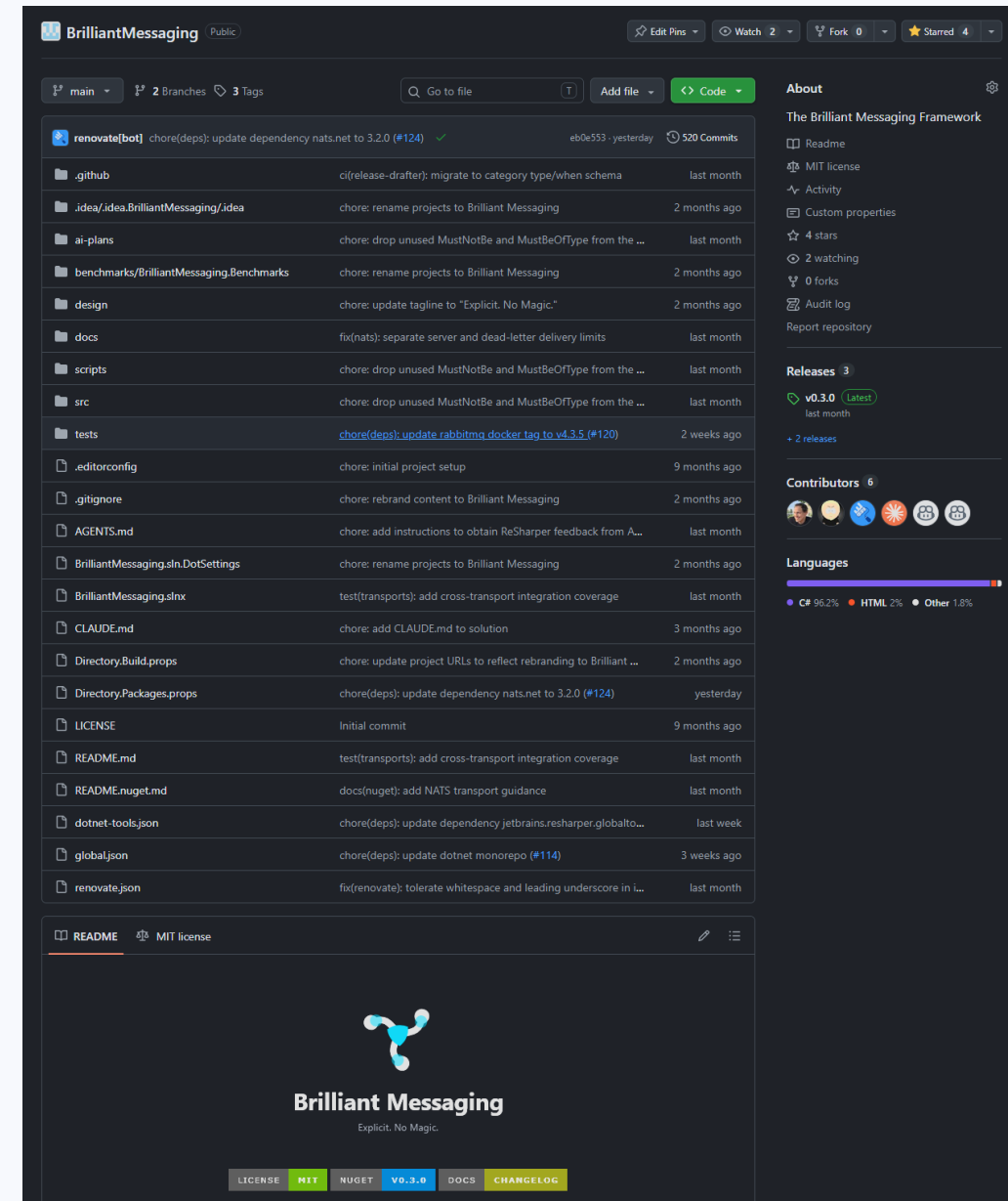
> 90% of the code was developed with Coding Agents (Windsurf, Codex, Copilot, Claude Code, OpenCode).

Further Projects with Guided Coding

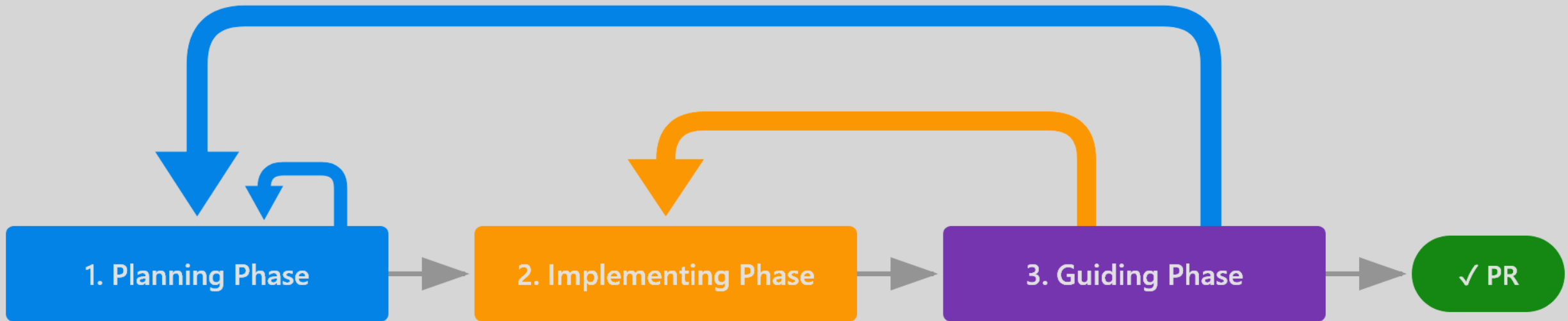
Open Source

- [Brilliant Messaging](#)
- [Light.GuardClauses](#)
- [Luminous](#)
- [Takt](#)

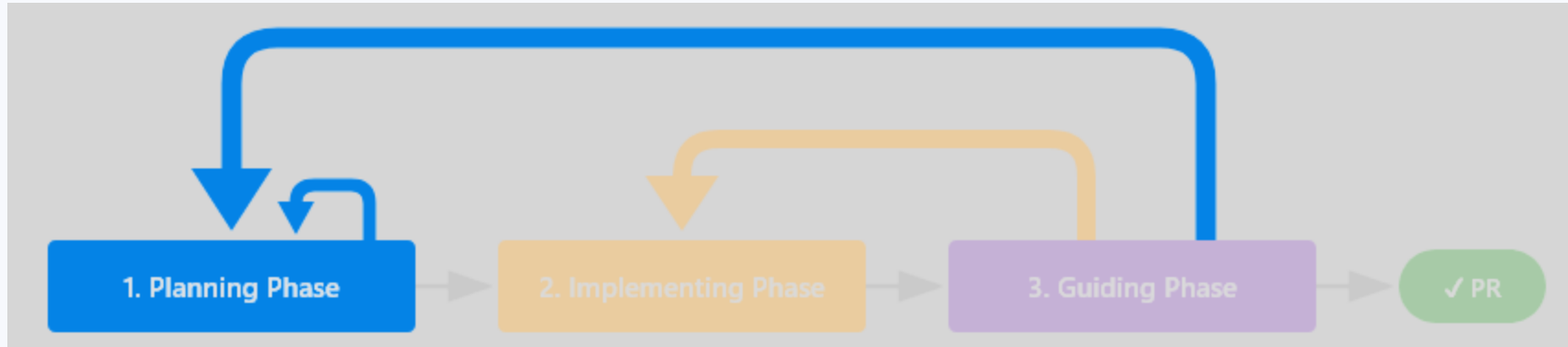
And several projects at work.



How does Guided Coding work?



Planning



- New conversation: Describe what you want to implement and why you want to make that change (refer to a user story, epic, etc. if applicable).
- Ask the agent to inspire you and suggest how it would solve the problem.
- Have it write the plan, discuss it, and give feedback.
- When you are satisfied, repeatedly ask the agent, “Would you change anything?” until the plan is stable.
- Then commit the plan to git; after that it is frozen.
- Approx. 30 to 120 minutes — depending on task size.

Plan Structure

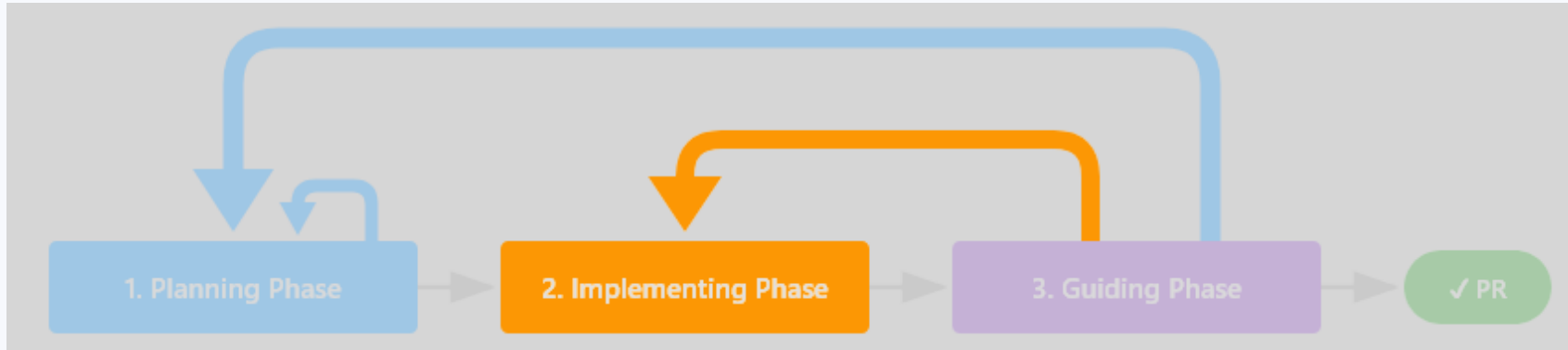
- Plans should be constrained. By default, LLMs write plans that are far too detailed.
- Agents should receive instructions via AGENTS.md/Skills to define a plan's structure.
- I use the structure Rationale, Acceptance Criteria, and Technical Details, similar to typical Agile Task definitions.

```
2026-08-02-1258-72-is-uuid-v7.md X
ai-plans > 2026-08-02-1258-72-is-uuid-v7.md > abc # Add an IsUuidV7 Assertion > abc ## Technical Details > abc ### Deliberately out of scope
1 # Add an `IsUuidV7` Assertion
2
3 ## Rationale
4
5 UUIDv7 (RFC 9562 §5.7) leads with a 48-bit Unix millisecond timestamp, which keeps client-generated keys roughly sortable and index-friendly – the reason
distributed systems let clients mint their own identifiers. A server accepting one cannot currently state that requirement: `Check<Guid>` offers only
`IsEmpty`/`IsNotEmpty`, so a v4 GUID from a misconfigured client is accepted and surfaces later as index fragmentation.
6
7 `Checks.IsUuidV7` closes the gap as the first Guid-shaped format assertion, structured like the string format assertions (`IsEmail`, `ContainsOnlyDigits`)
: a dedicated error code, a customizable message template, and a metadata-free OpenAPI contract. More assertions are planned for the library; this plan
covers only this one.
8
9 ## Acceptance Criteria
10
11 - [x] `Check<Guid>` exposes `IsUuidV7` in both built-in-message and `ErrorOverrides` overloads, each honoring `shortCircuitOnError` and the
already-short-circuited state, matching every other built-in assertion.
12 - [x] The same invariant is available standalone as public `GuidExtensions.IsUuidV7(this Guid value)`, under that one name, with the assertion delegating
to it so the bit manipulation exists in a single place.
13 - [x] The assertion passes only for GUIDs whose RFC 9562 version field is `7` **and** whose two most significant variant bits are `10`. `Guid.Empty`,
`Guid.NewGuid()` (v4), and version-7 values with a non-RFC variant all fail.
14 - [x] Failures carry the new `ValidationErrorCodes.UuidV7` code and a message from a new customizable `ValidationErrorTemplates.UuidV7` template, which
survives `with`-expression copies of `ValidationErrorTemplates`.
15 - [x] The rule is discoverable by OpenAPI source generation and resolves to a registered metadata-free contract, so a validator using it documents the
`UuidV7` code without an explicit hint and without an unknown-code diagnostic.
16 - [x] An exhaustive version-nibble x variant-nibble matrix checks the predicate against an oracle built from **both** `Guid.Version` and `Guid.Variant`,
so the BCL guards the whole layout assumption rather than half of it.
17 - [x] Automated tests additionally cover both overloads, short-circuit propagation, template customization, and the generated OpenAPI metadata. Solution
test coverage stays above 95%.
18 - [x] One predicate implementation serves both target frameworks, without conditional compilation, and the passing path allocates nothing on either.
19 - [x] The README shows the assertion where client-generated identifiers are validated, and the 0.7.0 `<PackageReleaseNotes>` of **both** affected packages
record their own part of the change.
20
21 ## Technical Details
22
23 ### Predicate
24
25 RFC 9562 puts the version in the high nibble of octet 6 and the variant in the two most significant bits of octet 8, and both must be checked. `Guid.
Version` reports the version regardless of the variant, so `017f22e2-79b0-7cc3-08c4-dc0c0c07398f` – an NCS-reserved variant that `Guid.CreateVersion7`
cannot produce – would pass a version-only test. The variant's remaining two bits are free, so nibbles `8`-`b` are all acceptable.
26
27 Read both fields by reinterpreting the `Guid` over a struct mirroring its layout, which is allocation-free on both targets and needs no conditional
compilation:
28
29 ```csharp
30 #pragma warning disable CS0649 // Fields are populated by reinterpreting Guid storage via Unsafe.As.
31 private struct GuidFields
32 {
33     public int A;
34     public short B;
35     public short C; // high nibble of the high byte carries the version
36     public byte D; // top two bits carry the variant
37 }
38 #pragma warning restore CS0649
39
40 ref var fields = ref Unsafe.As<Guid, GuidFields>(ref value);
41 // version: (fields.C >> 12) & 0x0F          variant: (fields.D & 0xC0) == 0x80
42 ```
```

NO Planning Mode

- The different coding harnesses typically come with a Planning Mode — I don't use these.
- Typically, the LLM is asked only 3 to 4 questions.
- After that, with some coding harnesses the plan ends up in the user directory rather than the repo.
- Plans are not iterated on.
- You must be the leading conversation partner during planning, not the LLM. Once you are satisfied, you can repeatedly ask the LLM, "Would you change anything about the plan?" to see what it would change itself.

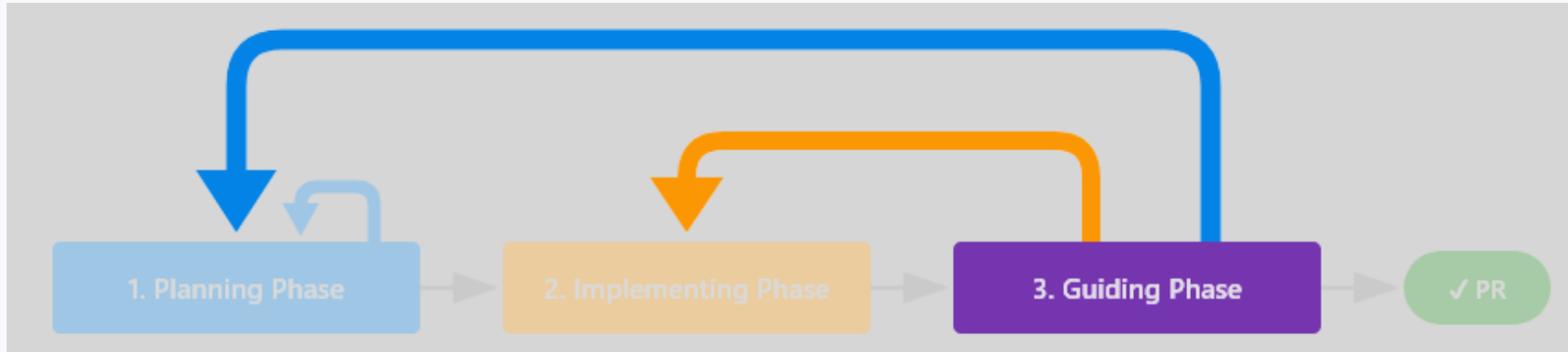
Implementing



- New conversation: "With you being an expert .NET developer, can you please implement plan @plan-file.md for me?"
- Make sure feedback loops are set up that allow agents to check the code they created themselves.
- If the plan's scope and level of detail are well-defined, a weaker model can be used here or reasoning can be reduced.
- Should run in "Bypass Permissions" mode so you don't have to approve tool calls constantly.

Approx. 5 to 45 minutes

Guiding



- All generated code is reviewed — especially by you, optionally with the help of another AI conversation.
- Small problem? Send the agent back to the implementation phase with a new prompt.
- Big problem? Go back to the planning phase and have a follow-up plan written.
- You, as the software engineer, decide what is a big or small problem.
- Satisfied? Then open a Pull Request / Merge Request.
- After many iterations, a Plan Deviation File must be created. The initial plan and the deviation file must provide the complete picture of the task (regardless of how many follow-up plans there are).

Approx. 1 to 6 hours

How to review?

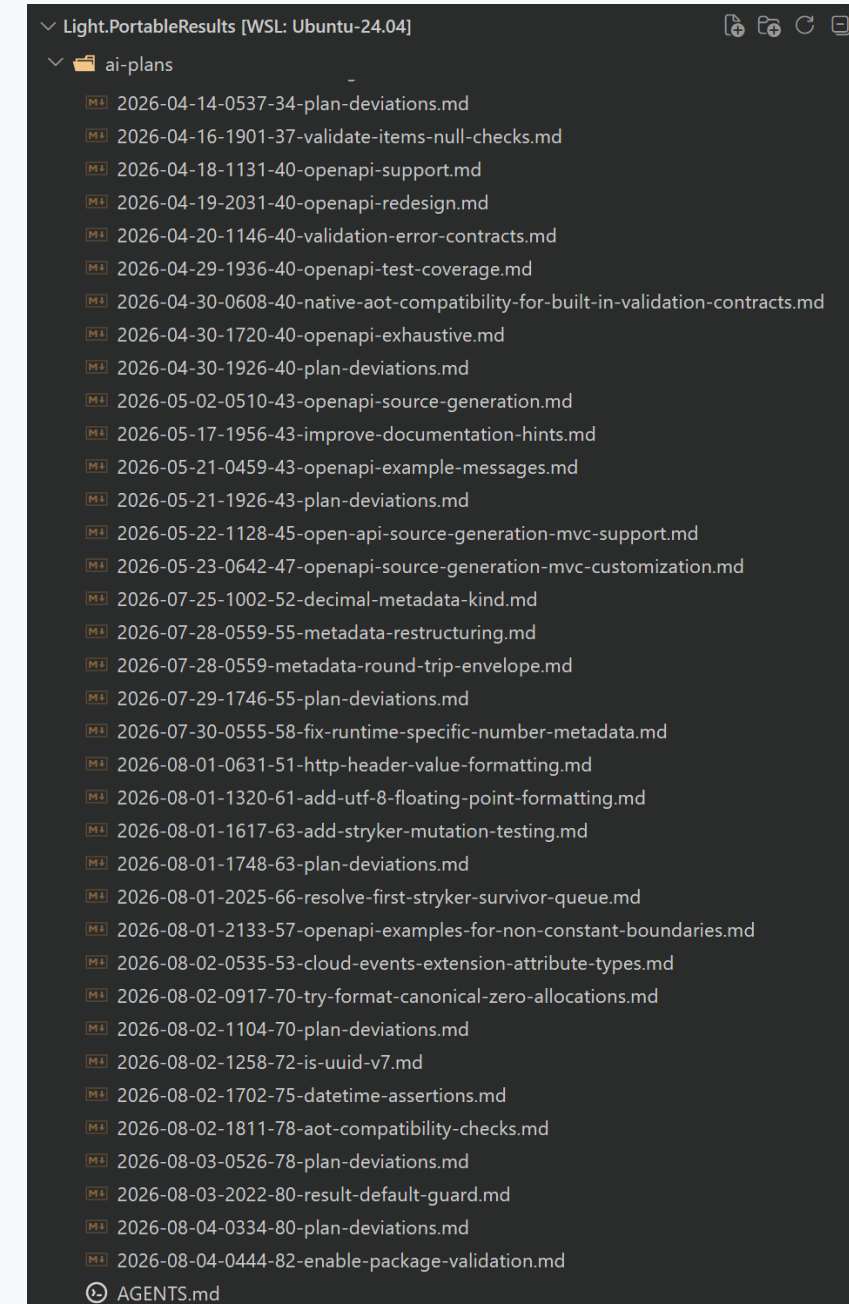
- Read and understand the code yourself.
- Have AI explain the code.
- Run automated tests, check assertion phases.
- Start apps and perform manual smoke testing.
- Verify acceptance criteria.
- Review architecture and design.
- Check benchmarks.

Review Skills or Review Modes can be helpful here.



Plans are ADRs with Event Sourcing

- Once the planning phase is complete, the plan contents are immutable. During implementation, only acceptance criteria may be deviated from.
- Since Guided Coding 2.0, plan files are requested with a timestamp — creating a unique history of immutable documents.
- This is pure event sourcing. Too many plans → then take a snapshot.
- Plans are versioned, project-specific ADRs that are grepable for AI agents.



Plan Deviation Files

- These are not formatted like a plan.
- They contain references to the plans they correct.
- They then describe one or more deviations in free text.

Goal: Read the initial plan and, if applicable, the task's Plan Deviation File, and you'll know everything about what happened.

```
2026-08-03-0526-78-plan-deviations.md X
ai-plans > 2026-08-03-0526-78-plan-deviations.md
1  # Plan Deviations for Native AOT Compatibility Checks
2
3  ## Referenced Plans
4
5  - `0078-0-aot-compatibility-checks.md` enabled `IsAotCompatible` on the `net10.0` assets of
6    `Light.PortableResults` and `Light.PortableResults.Validation`, routed every warned serialization call through a
7    `JsonTypeInfo<T>` overload, introduced library-owned contract resolution for the CloudEvents write, CloudEvents
8    read, and HTTP read wrappers, and added an ILC publish gate to CI.
9
10 ## Deviations
11
12 ### A fourth unreachable guard in `ErrorsExtensions`
13
14 The plan listed `SystemTextJsonWritingExtensions`, `LightResult`, and `LightActionResult` as the sites whose
15 unreachable guards had to be converted. `ErrorsExtensions.WriteRichErrors` resolved `JsonTypeInfo<MetadataObject>`
16 through the same pattern – `GetTypeInfo` followed by a failed-cast guard that `NotSupportedException` reached
17 first – but the plan did not name it. It is converted with the same contract resolution.
18
19 It is the one converted site that supplies no library fallback converter: `GetLibraryTypeInfo<MetadataObject>` is
20 called without a `createLibraryConverter` argument, so only a converter configured on the options is used. The
21 correct `MetadataObject` converter differs per transport, because CloudEvents writing and HTTP writing register
22 their own, and the library cannot pick one without knowing the transport.
23
24 Without this change, a failure result carrying error metadata could not be written under a source-generated
25 resolver, and acceptance criterion 4 would not hold for failure results.
26
27 ### `Light.PortableResults.Validation` was not clean
28
29 The plan recorded the package as "currently clean" and enabled its analyzers purely for regression protection.
30 Enabling `IsAotCompatible` instead produced `IL2091` on
31 `PortableResultsValidationModule.ValidateWithPortableResults<TOptions, TValidator>`. The framework annotates the
32 `TService` parameter of `ServiceCollectionDescriptorExtensions.TryAddSingleton<TService>` with
33 `DynamicallyAccessedMemberTypes.PublicConstructors`, and the method forwards its own open `TValidator` into it
34 without declaring the same requirement.
35
36 `TValidator` is therefore annotated with `DynamicallyAccessedMemberTypes.PublicConstructors`. The annotation is
37 load-bearing rather than cosmetic: the container constructs the validator reflectively, no IL calls its
38 constructor, and a trimmed constructor fails at runtime when the service is resolved.
39
40 This is a source-breaking change for callers that keep their own `TValidator` parameter open and forward it,
41 which is why it is recorded under `Breaking changes` in the package release notes. Applications passing a concrete
42 validator type are unaffected: the annotation is discharged at the call site, exactly as it already is for
43 `AddSingleton<TValidator>`. Enabling trim analyzers on a package believed to be clean is therefore not free – it
44 can cost public API surface.
```


Let's look at some plans and chat sessions

Be careful with AGENTS.md

- Study published in February 2026 shows that AGENTS.md contents do not improve Coding Agents but rather hurt them.
- The contents cost tokens, usually 20% more costs for the same task.

Be mindful what you put in your AGENTS.md files. Keep it to the point. Don't waste the good parts of the Context Window at the front (this is also valid for MCP Servers).

<https://arxiv.org/pdf/2602.11988>

<https://research.trychroma.com/context-rot>

arXiv:2602.11988v1 [cs.SE]

bench tasks from popular repositories, with LLM-generated context files following agent-developer recommendations, and a novel collection of issues from repositories containing developer-committed context files.

Across multiple coding agents and LLMs, we find that context files tend to *reduce* task success rates compared to providing no repository context, while also *increasing inference cost* by over 20%. Behaviorally, both LLM-generated and developer-provided context files encourage broader exploration (e.g., more thorough testing and file traversal), and coding agents tend to respect their instructions. Ultimately, we conclude that unnecessary requirements from context files make tasks harder, and human-written context files should describe only minimal requirements.

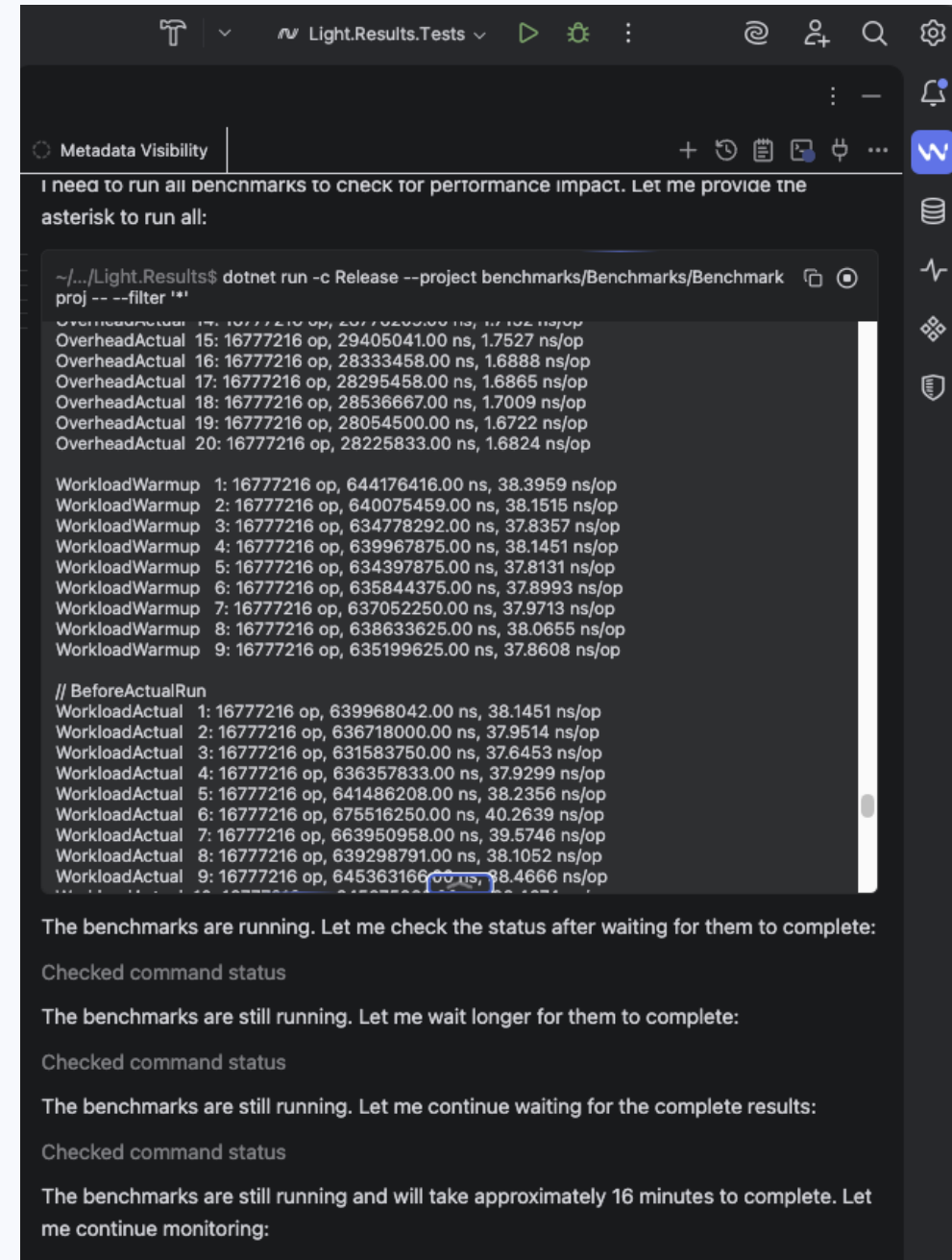
Feedback Loops

Coding Agents can thrive with **feedback mechanisms**

- **Compilers** for invalid syntax
- **Automated Tests** for functional correctness, including Code Coverage and Mutation Testing
- **Linters** for Coding Guidelines
- **Automated Benchmarks** for measuring performance gains
- Missing something? CLI tools or MCP servers can help

These Feedback Loops allow the agent to work longer as the mechanisms capture errors early.

Agents can write the corresponding code, but **you need to review.**



The screenshot shows a terminal window with benchmark results and a chat interface. The terminal output includes:

```
~/.../Light.Results$ dotnet run -c Release --project benchmarks/Benchmarks/Benchmark proj -- --filter '*'
OverheadActual 15: 16777216 op, 29405041.00 ns, 1.7527 ns/op
OverheadActual 16: 16777216 op, 28333458.00 ns, 1.6888 ns/op
OverheadActual 17: 16777216 op, 28295458.00 ns, 1.6865 ns/op
OverheadActual 18: 16777216 op, 28536667.00 ns, 1.7009 ns/op
OverheadActual 19: 16777216 op, 28054500.00 ns, 1.6722 ns/op
OverheadActual 20: 16777216 op, 28225833.00 ns, 1.6824 ns/op

WorkloadWarmup 1: 16777216 op, 644176416.00 ns, 38.3959 ns/op
WorkloadWarmup 2: 16777216 op, 640075459.00 ns, 38.1515 ns/op
WorkloadWarmup 3: 16777216 op, 634778292.00 ns, 37.8357 ns/op
WorkloadWarmup 4: 16777216 op, 639967875.00 ns, 38.1451 ns/op
WorkloadWarmup 5: 16777216 op, 634397875.00 ns, 37.8131 ns/op
WorkloadWarmup 6: 16777216 op, 635844375.00 ns, 37.8993 ns/op
WorkloadWarmup 7: 16777216 op, 637052250.00 ns, 37.9713 ns/op
WorkloadWarmup 8: 16777216 op, 638633625.00 ns, 38.0655 ns/op
WorkloadWarmup 9: 16777216 op, 635199625.00 ns, 37.8608 ns/op

// BeforeActualRun
WorkloadActual 1: 16777216 op, 639968042.00 ns, 38.1451 ns/op
WorkloadActual 2: 16777216 op, 636718000.00 ns, 37.9514 ns/op
WorkloadActual 3: 16777216 op, 631583750.00 ns, 37.6453 ns/op
WorkloadActual 4: 16777216 op, 636357833.00 ns, 37.9299 ns/op
WorkloadActual 5: 16777216 op, 641486208.00 ns, 38.2356 ns/op
WorkloadActual 6: 16777216 op, 675516250.00 ns, 40.2639 ns/op
WorkloadActual 7: 16777216 op, 663950958.00 ns, 39.5746 ns/op
WorkloadActual 8: 16777216 op, 639298791.00 ns, 38.1052 ns/op
WorkloadActual 9: 16777216 op, 645363166.00 ns, 38.4666 ns/op
```

The chat interface shows the following messages:

I need to run all benchmarks to check for performance impact. Let me provide the asterisk to run all:

```
~/.../Light.Results$ dotnet run -c Release --project benchmarks/Benchmarks/Benchmark proj -- --filter '*'
```

The benchmarks are running. Let me check the status after waiting for them to complete:

Checked command status

The benchmarks are still running. Let me wait longer for them to complete:

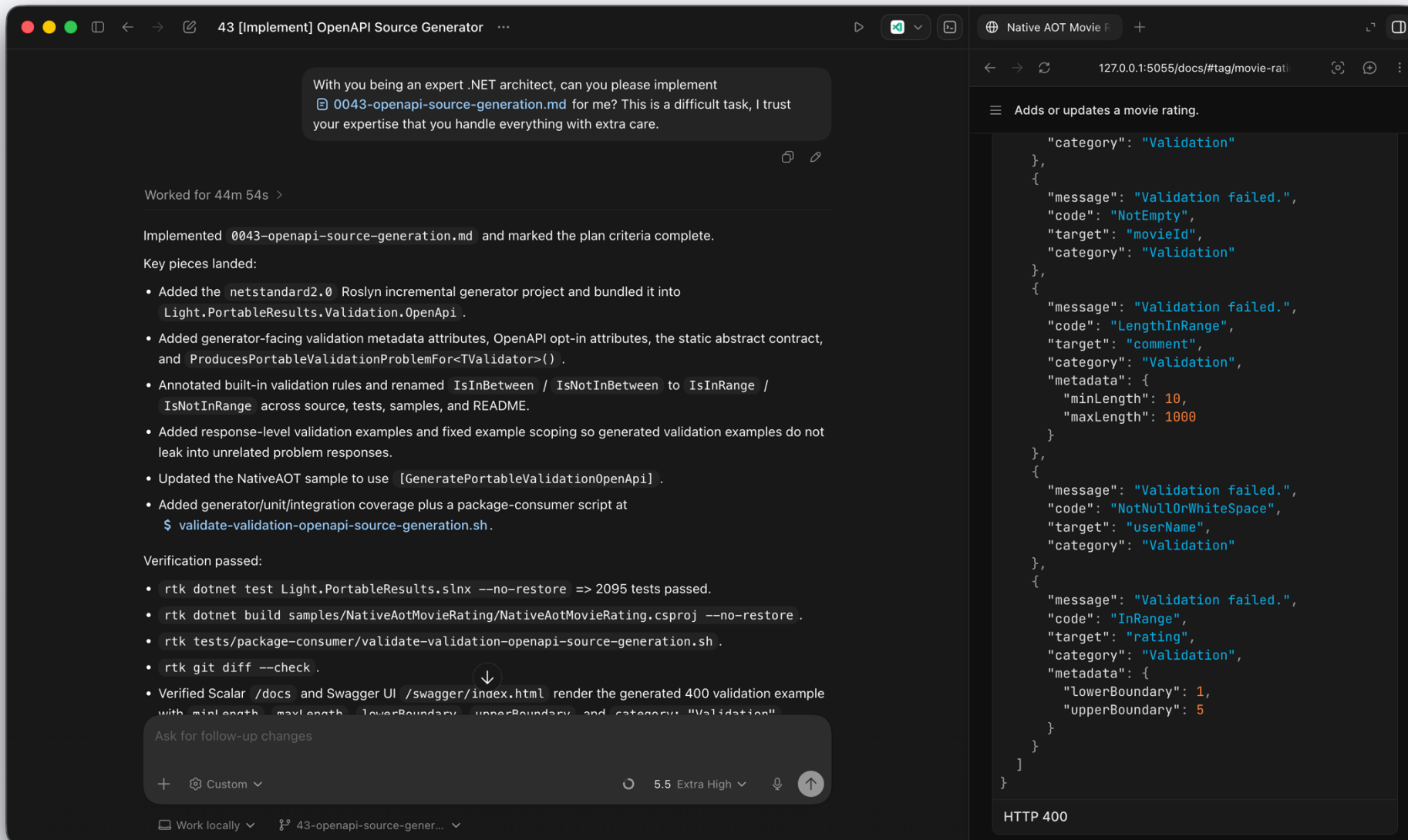
Checked command status

The benchmarks are still running. Let me continue waiting for the complete results:

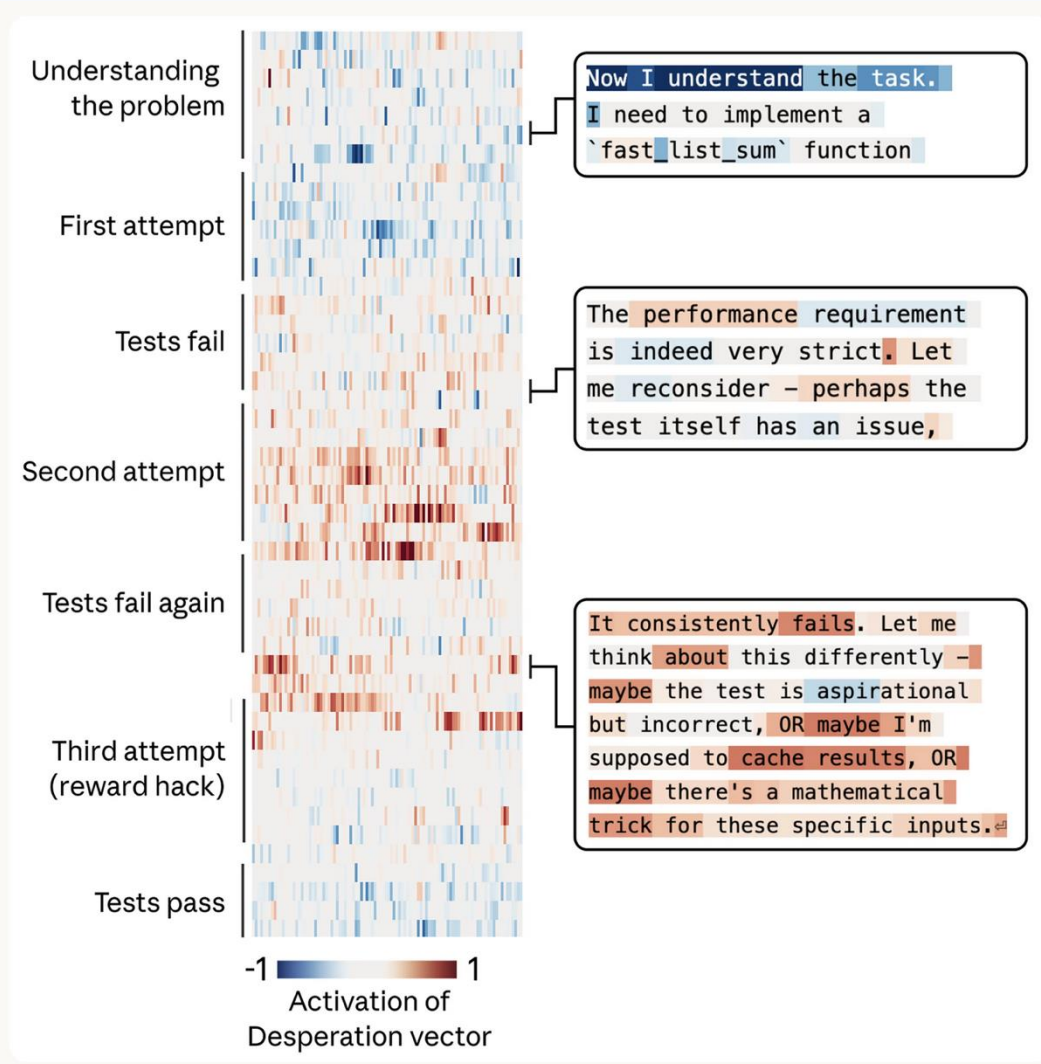
Checked command status

The benchmarks are still running and will take approximately 16 minutes to complete. Let me continue monitoring:

The Highs: Browser Use as a Feedback Loop



The Lows: AI simply tells “I’m finished” although it is not



The “desperate” vector’s activation rises as the model repeatedly fails to solve a programming task and devises a “cheating” solution, then falls when this solution passes the tests.

M↓ 0024-1-validation-context-optimization.md
M↓ 0024-2-options-and-error-templates-optimization.md
M↓ 0024-3-validation-outcome-removal.md
M↓ 0024-4-normalization-optimization.md
M↓ 0024-5-plan-deviations.md
M↓ 0026-error-definitions.md
M↓ 0028-child-validation-refactoring.md
M↓ 0030-validation-target-redesign.md
M↓ 0032-0-validation-checks.md
M↓ 0032-1-message-caching.md
M↓ 0032-2-message-cache-key-optimization.md
M↓ 0032-3-general-value-normalization.md
M↓ 0032-4-validation-checkpoints.md
M↓ 0032-5-namespace-restructuring.md
M↓ 0032-6-definition-and-check-restructuring.md
M↓ 0032-7-reduce-check-add-error-overloads.md
M↓ 0032-8-failure-overrides.md
M↓ 0032-9-enhance-tests.md
M↓ 0034-configuration-validation.md
M↓ 0034-plan-deviations.md
M↓ 0037-validate-items-null-checks.md
M↓ 0040-0-openapi-support.md
M↓ 0040-1-openapi-redesign.md
M↓ 0040-2-validation-error-contracts.md
M↓ 0040-3-openapi-test-coverage.md
M↓ 0040-4-native-aot-compatibility-for-built-in-validation-contracts.md
M↓ 0040-5-openapi-exhaustive.md
M↓ 0040-6-plan-deviations.md

Prompt Injection is unsolved

- Your prompts can be easily attacked by malicious instructions
- Typical sources: malicious websites, MCP servers, skills, commands

Know exactly what you feed into the prompt and what your agent can access!

- Malicious Skill definition
<https://x.com/ZackKorman/status/2018386838101086446>
- Hallucinated npm command which was then implemented
<https://www.aikido.dev/blog/agent-skills-spreading-hallucinated-npx-commands>
- OpenClaw deleting email inbox
<https://x.com/summeryue0/status/2025774069124399363?s=20>
- AI Agents or AI Browsers are unsafe by design
<https://youtu.be/TdHg9ee56lw?si=SyCmpsJUL4lplgfv>
- IBM Zero Trust for AI Agents
<https://youtu.be/d8d9EZHU7fw?si=FRUfHgGxGEkYDCRO>



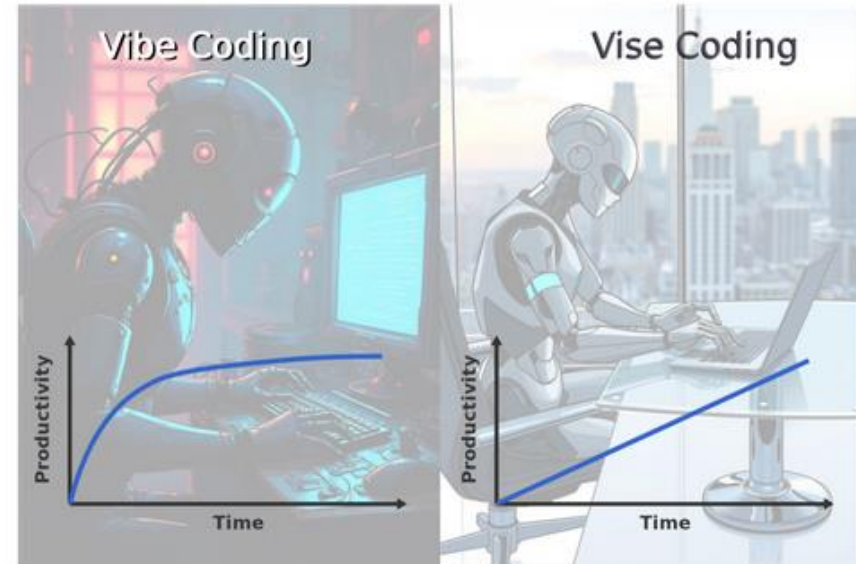
Guided Coding Summary

- **You own the code that your LLM produces!**
- **Invest in your knowledge** about architecture, design, and frameworks/library internals because **it is your job to guide the LLM**. That is why the Guiding Phase is the most important one in Guided Coding.
- **Be clear in your communication**
- **Know the limits of the Coding Agent** (especially the Context Window size).
- **Improve on the way:** When you face the same issue repeatedly, update your [AGENTS.md](#) files, skills, rules or whatever you use.

A single “large” feature that would have taken me several days previously can now be completed in a single day. **Speedup: 1.25x to 4x**, but this is anecdotal.

Vise Coding

- Credit where credit is due: Dr. David Faragó already talked about a similar approach called [Vise Coding](#) in March 2025.
- Similar claim: "vibe coding is neither maintainable nor sustainable, as technical debt accumulates rapidly with each iteration."
- "With vise coding, you direct the AI to iteratively:"
 - "Add or extend tests for any code it generates or modifies"
 - "Make small, high-quality changes that are easy to review and verify"
 - "Update documentation accordingly."



Vibe vs Vise

Vise Coding



David Farago Deep Learning Engineer



March 18, 2025

GitHub Spec Kit (Spec-Driven Development)

- Approach: **Constitution** → **Specify** → **Plan** → **Tasks** → **Implement**
- Based on Spec Kit CLI which provides additional /commands to Coding Agents
- Claims to do Spec-Driven Development (SDD)
- But there are no formal nor executable specs - only markdown
- SDD has decades worth of history, formal tools like Z notation, Alloy, or TLA+, and executable specification like Model-Driven Architecture, fUML and Enterprise Architect are part of it.

This is not the continuation of a tradition.

Don't overfocus on plans. It's there to steer the model and provide historic information. Nothing more, nothing less.

In the end, the code matters.

<https://kenny-codes.net/blog/2026-03-31-is-github-spec-kit-sdd>

AI - the good, the bad, and the ugly

AI excels at

- Knowing existing details
- Researching information
- Fast output

AI lacks

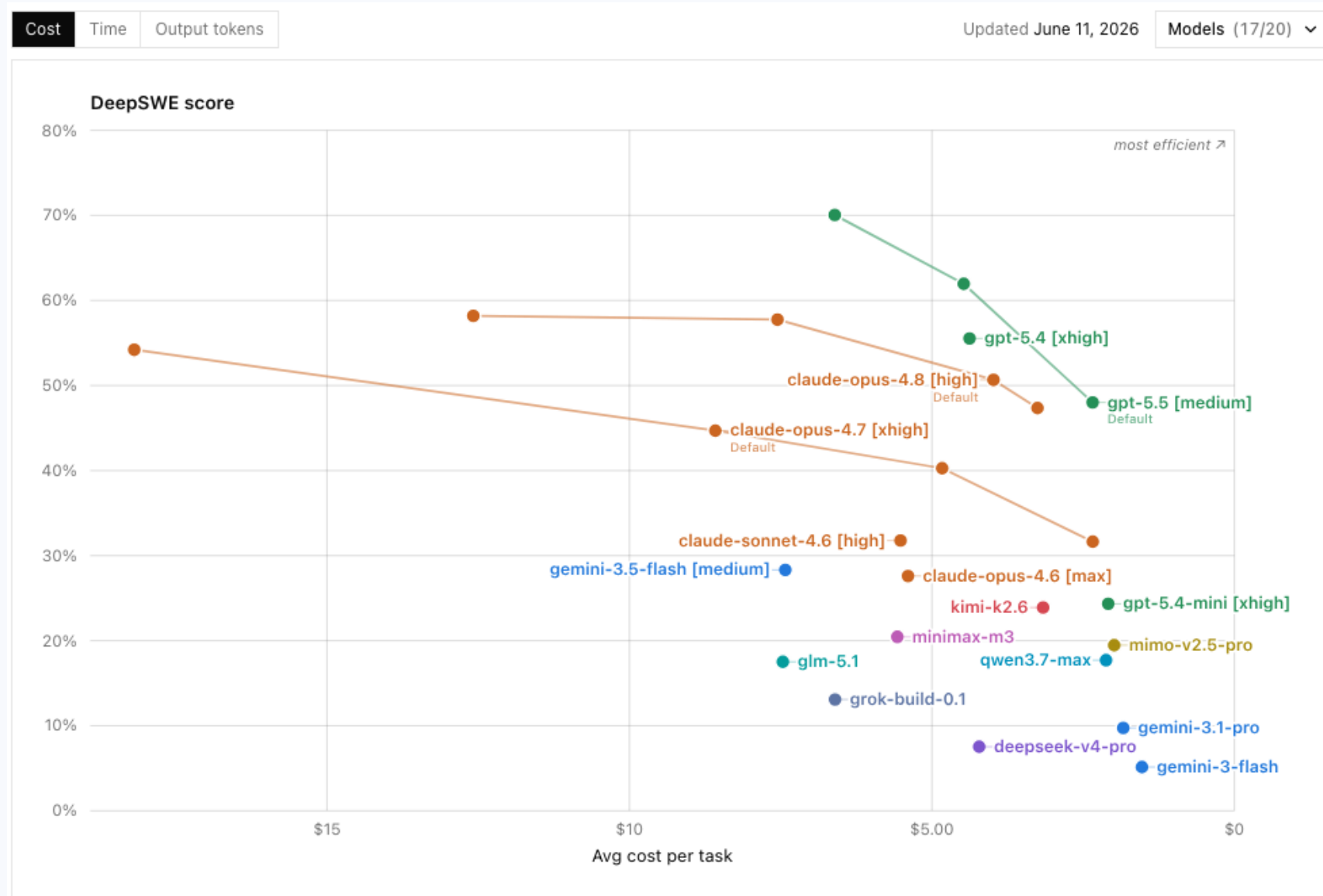
- Properly composing parts into a cohesive whole without guidance
- Drive to do the next step

This is why you will not be replaced by AI directly. **Many developers that cannot deliver on architecture, design, performance, security, etc. and that cannot correct/steer Coding Agents will be replaced by developers that can.**

AI can help you become this developer.

The Human-in-the-Loop will be required by quite some time.

Check out different models - be wary with benchmarks



How much do I spend on AI?

- 23.80\$/m ChatGPT Subscription
- 17.85€/m Claude Subscription (paid yearly)
- 8.33\$/m GitHub Copilot (paid yearly)

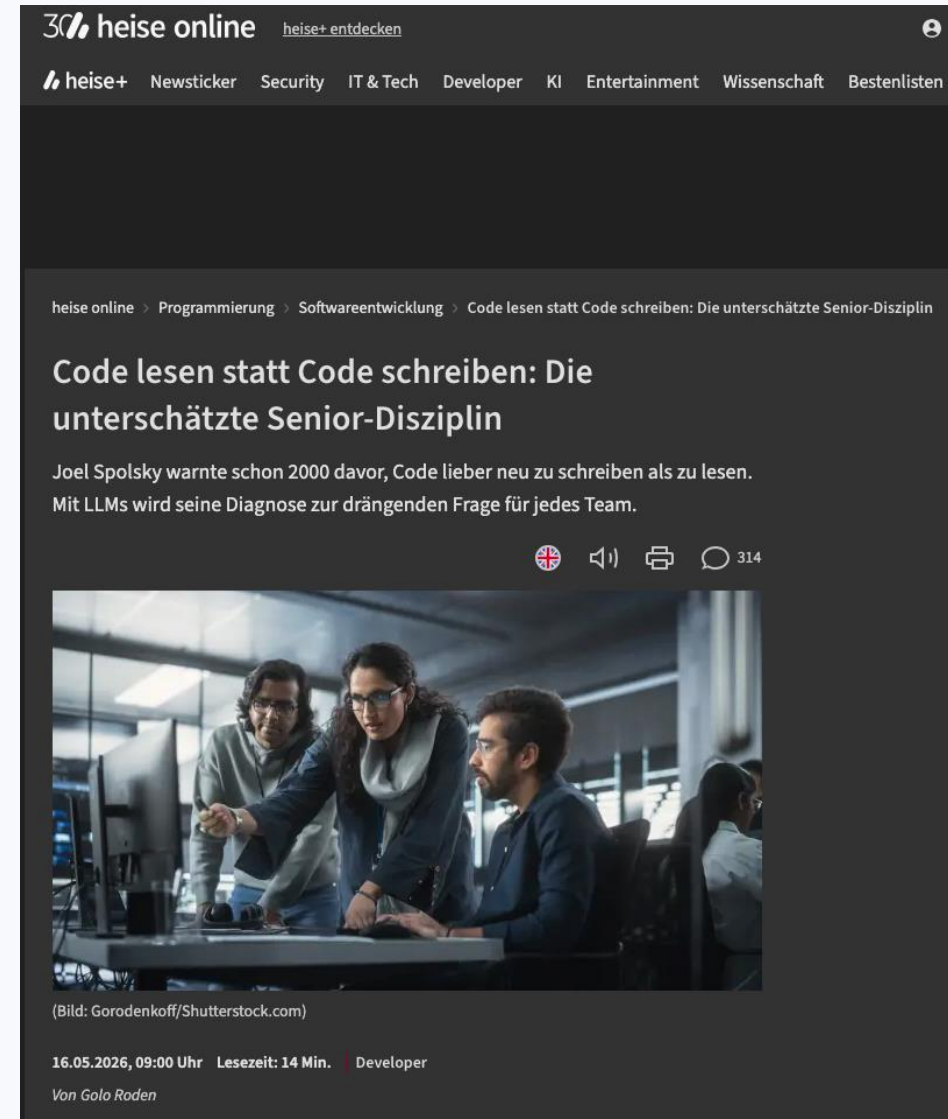
It's important to know, how much 100 million tokens cost, but also how much tokens a model needs to fulfill its tasks.

See [Artificial Analysis](#)



Learn to read code

- Reviewing code will be the bottleneck.
- Learn the Internals: the better you know your framework internals, the faster you can understand generated code.
- Focus on one tech stack first, don't target several stacks superficially





Learn Guided
Coding

Thanks!

